

1. URL Shortener

(e.g. TinyURL, Bitly)

"Turn long URLs into short, shareable links"

1. Requirements

- Shorten a long URL
- Redirect short URL → original URL
- Custom short URLs (optional)
- Track analytics (clicks, location, etc.)
- High availability & low latency

2. Scale (Rough Estimate)

- 100M short URLs per month
- Read : Write ratio = 100 : 1
- Low latency (< 100 ms)
- High availability (99.9%)

3. API Design

• POST /shorten

Request: { "url": "https://..." }

Response: { "shortUrl": "https://sho.rt/abc123" }

- GET /{shortCode} → Redirects to original URL

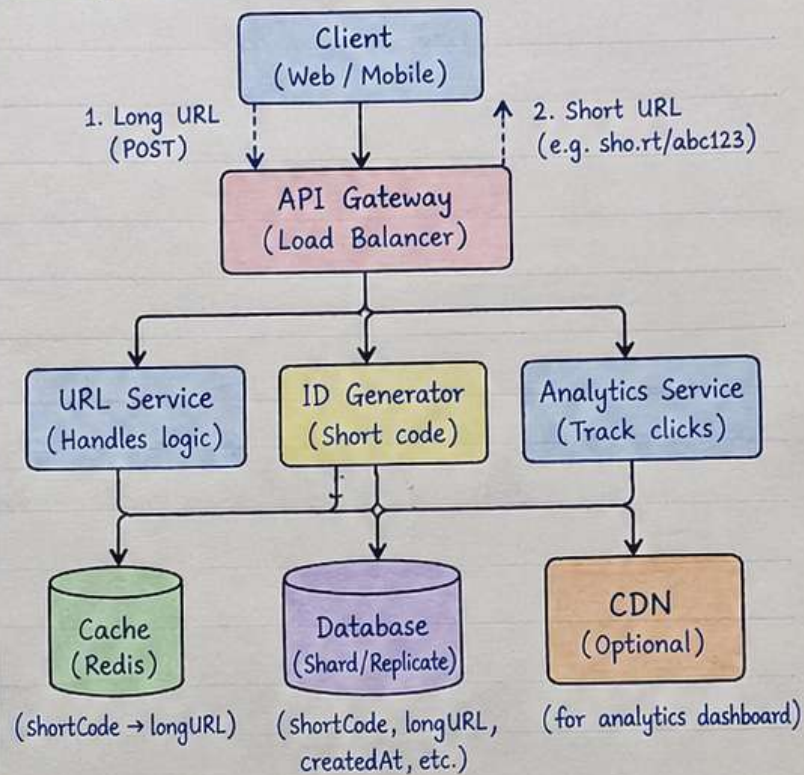
5. Short Code Generation

- Generate a unique ID for each URL
- Use Base62 encoding (0-9, a-z, A-Z) for shorter length
- Methods:
 - Auto-increment ID (e.g. 1001 → 7Lq)
 - Distributed ID generator (e.g. Twitter Snowflake)
 - Hashing (e.g. MD5 + Base62)
- Ensure uniqueness (handle collisions)

6. Database Schema (Simplified)

id	short_code	long_url	created_at	clicks
1	7Lq	https://...	2024-01-01	0
2	aB9	https://...	2024-01-01	5
...	...	...	...	...

4. High Level Architecture



7. Redirect Flow

- ① User clicks on short URL (e.g. sho.rt/7Lq)
- ② Check cache (Redis) for long URL
- ③ If not in cache → fetch from DB
- ④ Increment click count (async)
- ⑤ Return 301/302 redirect to original URL

8. Key Challenges

- Generate unique, short IDs at scale
- Handle high read traffic (use caching + CDN)
- Prevent abuse (spam/malicious links)
- Analytics (real-time & batch)
- Custom short URLs, expiry, rate limiting

9. Scaling Strategies

- Use distributed ID generator
- Cache shortCode → longURL in Redis
- Use CDN for faster redirects
- Database sharding & replication
- Asynchronous analytics (Kafka, SQS, etc.)
- Multi-region deployment



Interview Tip: Focus on trade-offs (short code generation, caching, consistency) and how you'll handle massive read traffic.

"Simple idea, massive scale!"

2. Instagram

(Photo & Video Sharing Social Platform)

"Capture
Share
Stay Connected"

1. Requirements

- Upload photos & videos
- View feed (posts from followed users)
- Like, comment, share, save
- Follow / unfollow users
- Search users, hashtags
- Support stories (temporary content)
- High availability & low latency
- Handle millions of users and huge media files

2. Scale (Rough Estimate)

- Users: 2B+ (MAU)
- Daily uploads: 100M+ photos/videos
- Read: Write ratio = 100 : 1
- Avg. media size: 500 KB (image),
5-50 MB (video)
- Daily storage: $100M \times 1 \text{ MB} \approx 100 \text{ TB}$
- High availability (99.99%)

3. API Design (Simplified)

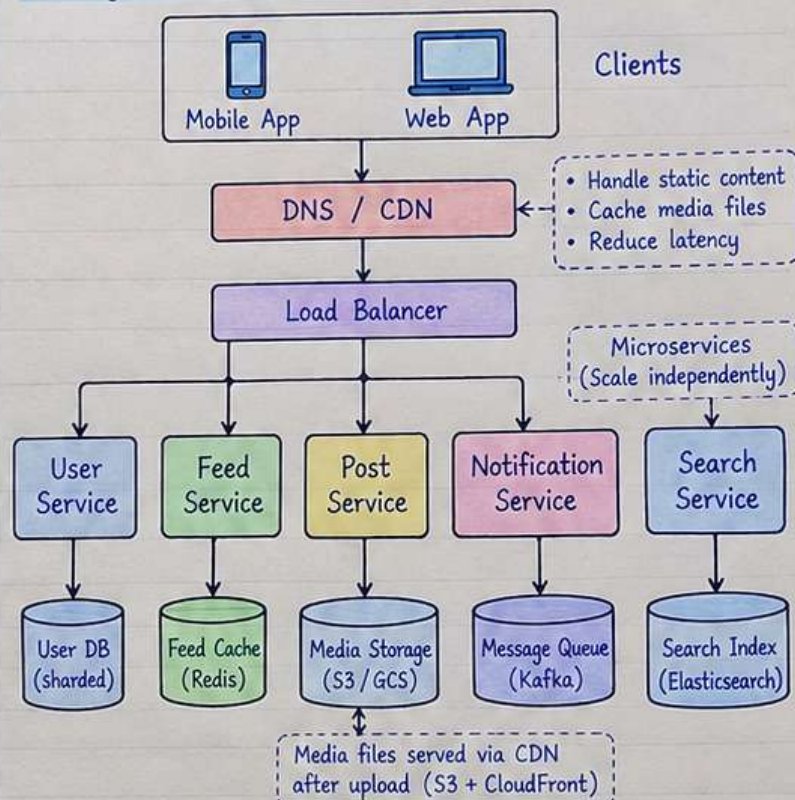
- POST /posts (upload photo/video)
- GET /feed (get home feed)
- POST /posts/{id}/like
- POST /posts/{id}/comment
- GET /users/{id} (user profile)
- POST /follow/{id}
- GET /search?q=... (users, hashtags)

4. Data Model (Simplified)

Users	Posts
id (PK)	id (PK)
username	user_id (FK)
email	media_url
password_hash	caption
profile_pic_url	type (image/video)
created_at	created_at

Followers	Likes	Comments
user_id (FK)	user_id (FK)	id (PK)
follower_id (FK)	post_id (FK)	post_id (FK)
created_at	created_at	user_id (FK)
		text
		created_at

5. High Level Architecture



6. Feed Generation (Key Part)

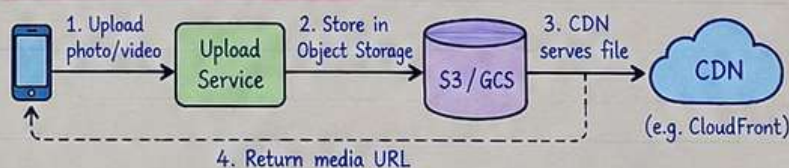
1. Fan-out on Write (Push)

- When a user posts, push to followers' feeds (pre-compute)
- Faster read, heavy write
- Good for normal users

2. Fan-out on Read (Pull)

- Fetch latest posts from followed users at read time
- Slower read, less write
- Used for celebrity accounts (millions of followers)
- Often a hybrid approach is used

7. Media Upload Flow



8. Key Challenges

- Handling massive media storage & bandwidth
- Real-time feed generation at scale
- Consistency (likes, comments, follows)
- Spam & abuse (fake accounts, inappropriate content)
- Low latency global access (use CDN, caching)
- Data privacy & content moderation
- Supporting new features (Reels, Stories, Live, etc.)



Interview Tips

- Focus on feed generation design (it's the core)
- Discuss trade-offs (push vs pull, consistency vs latency)
- Talk about scaling media storage (CDN, partitioning)
- Mention real-world optimizations (caching, sharding, queues)

"It's not just
a photo app,
it's a global
social system"

"Good system design connects people" ❤️

3. WhatsApp

(Real-time Messaging Platform)

"Simple Messages
Stronger Connections"

1. Requirements

- 1:1 messaging
- Group messaging
- Real-time delivery
- Online / offline status
- Read receipts (✓✓)
- Support text, images, videos, files
- End-to-end encryption
- High availability & low latency
- Billions of users, global scale

2. Scale (Rough Estimate)

- Users: 2B+ (MAU)
- Messages per day: 100B+
- Concurrent connections: 10M+
- Avg msg size: ≈ 10 KB (text), ≈ 100 KB (media)
- Low latency: < 200 ms
- High availability: 99.99%

3. API Design (Simplified)

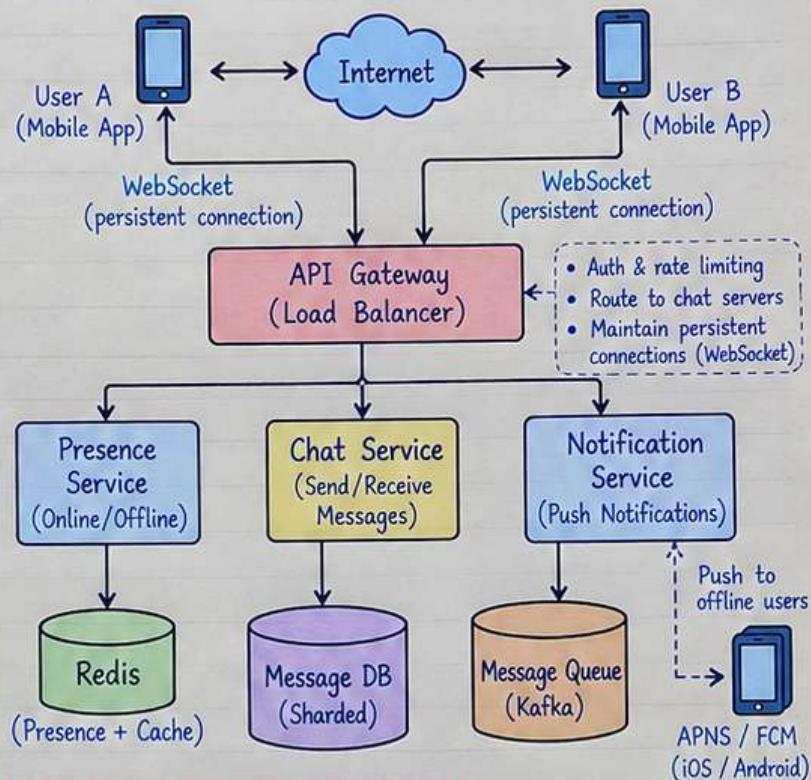
- POST /messages (send message)


```
{
    "to": "userId",
    "type": "text|image|video",
    "content": "..."
  }
```
- GET /messages?chatId=... (fetch messages)
- POST /messages/{id}/read (mark as read)
- GET /users/{id}/status (online/offline)
- POST /groups (create group)
- POST /groups/{id}/members (add/remove)

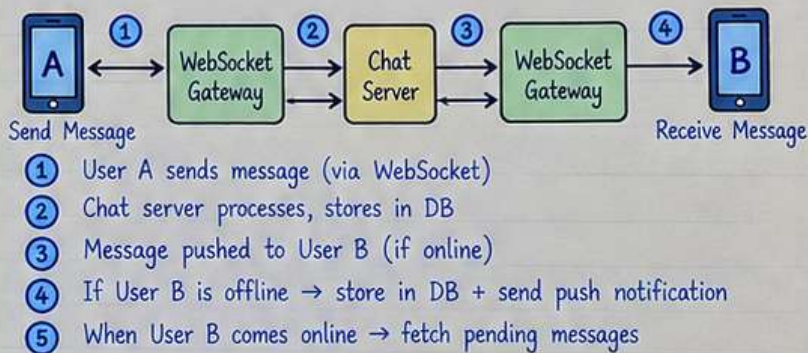
4. Data Model (Simplified)

Users	Messages	Conversations
id (PK)	id (PK)	id (PK)
phone_number	chat_id (FK)	type (1:1 / group)
name	sender_id (FK)	name (group)
profile_pic_url	type (text/image/...)	member_ids (array)
status (online/offline)	content (encrypted)	last_message_id
last_seen	timestamp	updated_at
created_at	status (sent/delivered/read)	

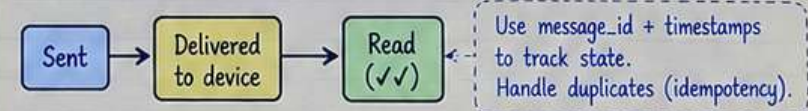
5. High Level Architecture



6. Real-time Message Flow



7. Message Delivery States



8. Key Challenges

- Scalability (billions of users, millions of concurrent connections)
- Message ordering (esp. in groups)
- Handling offline users
- End-to-end encryption (security + key management)
- Low latency & high availability
- Media/storage & large file handling
- Avoiding duplicate messages (idempotency)

9. Interview Tips

- Use WebSockets (vs polling)
- Discuss presence system (Redis)
- Talk about message queues (Kafka)
- How to handle ordering & retries
- Discuss end-to-end encryption
- Mention trade-offs (consistency vs availability)

 **Note:** Messages are end-to-end encrypted. Server stores encrypted content, so it cannot read messages.

"It's not just an app, it's a billion conversations." 😊

4. YouTube

(Video Sharing & Streaming Platform)

“Upload Process Stream to Billions”

1. Requirements

- Upload videos (any format/size)
- Process & convert to multiple resolutions
- Store and stream videos globally
- Search, like, comment, share
- Generate thumbnails, captions
- Support live streaming (optional)
- High availability & low latency
- Handle billions of videos and users

2. Scale (Rough Estimate)

- Users: 2B+ (monthly)
- Video uploads: 500+ hours per minute
- Total videos: 100B+
- Avg video size: 100 MB – 5 GB
- Read : Write ratio $\approx 1000 : 1$
- High availability (99.99%)
- Global scale (multi-region)

3. API Design (Simplified)

- POST /videos (upload video)
 - multipart upload (for large files)
 - returns { videoId, uploadUrl }
- GET /videos/{id} (metadata)
- GET /videos/{id}/stream (get stream URL)
- POST /videos/{id}/like
- POST /videos/{id}/comment
- GET /search?q=... (search videos)

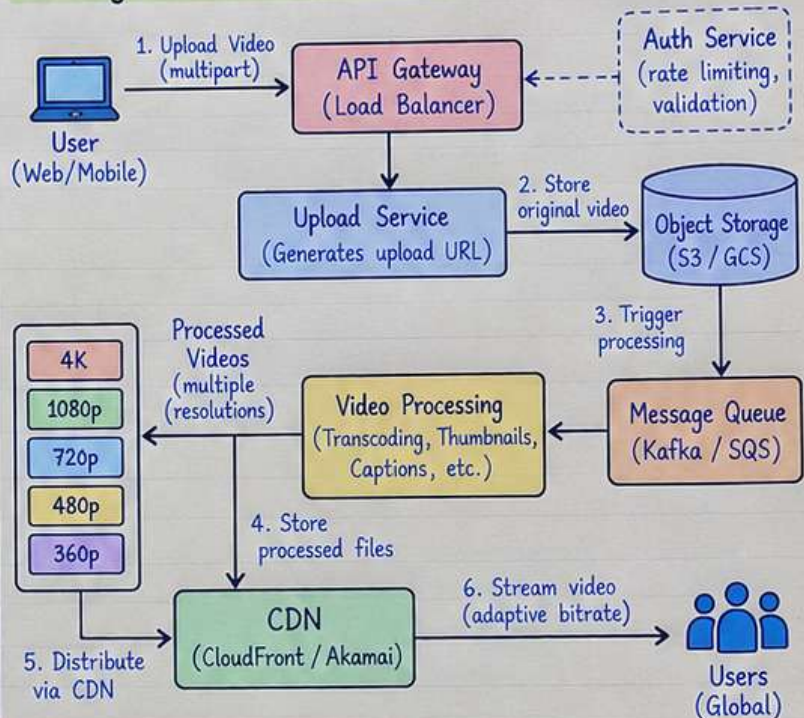
4. Data Model (Simplified)

Videos	Video Files
id (PK)	id (PK)
user_id (FK)	video_id (FK)
title	resolution (360p, 720p, 1080p, 4K)
description	file_url (S3/CDN)
thumbnail_url	file_size
status (uploading/processed)	codec
duration	created_at
views_count	
created_at	
updated_at	

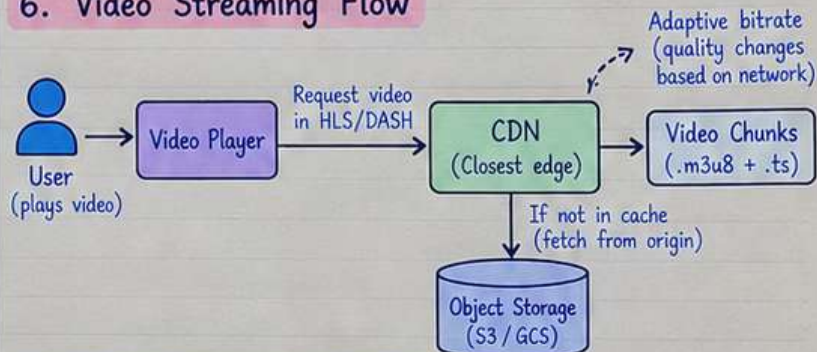
Comments	Likes
id (PK)	id (PK)
video_id (FK)	video_id (FK)
user_id (FK)	user_id (FK)
content	created_at
created_at	

Metadata in DB, actual video files in Object Storage (S3/GCS)!

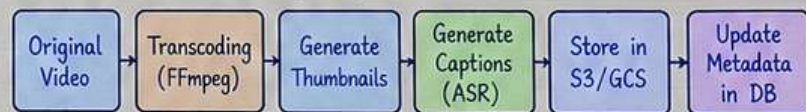
5. High Level Architecture



6. Video Streaming Flow



7. Video Processing Pipeline



8. Key Challenges

- Handling massive storage (petabytes)
- Efficient video transcoding (time + cost)
- Low latency streaming (global users)
- Adaptive bitrate streaming (different networks)
- Content moderation (copyright, inappropriate content)
- Search at scale (billions of videos)
- Hot vs cold videos (caching strategy)
- High bandwidth cost
- Live streaming (real-time processing)

9. Scaling Strategies

- Use CDN (edge caching)
- Store videos in object storage
- Asynchronous processing (queues)
- Auto-scaling transcoding workers
- Partition metadata DB (by video_id)
- Use cache for hot metadata
- Use microservices (upload, process, search, user, etc.)
- Multi-region deployment



Interview Tips:

- Discuss trade-offs: storage cost vs quality vs processing time
- How to handle new video formats?
- CDN, adaptive bitrate (HLS/DASH), and why not stream directly from origin?
- Talk about live streaming design (separate pipeline)
- Mention content moderation and abuse handling

“Scale video, not just views!”



5. Netflix N

(Video Streaming Platform)

"Entertainment
for a billion people
anywhere, anytime"

1. Requirements

- Stream movies, TV shows, documentaries
- Support multiple devices (TV, mobile, web)
- User profiles & personalized home page
- Search, watchlist, ratings, reviews
- Continue watching (resume playback)
- High availability & low latency (global)
- Support millions of concurrent users

2. Scale (Rough Estimate)

- Users: 250M+ (global)
- Concurrent streams: 10M+
- Total content: 100K+ titles
- Avg video size: 1 - 10 GB (per title)
- Read : Write ratio = 1000 : 1
- Low latency streaming (< 2 sec startup)
- High availability (99.99%)
- Global distribution (multi-region)

3. API Design (Simplified)

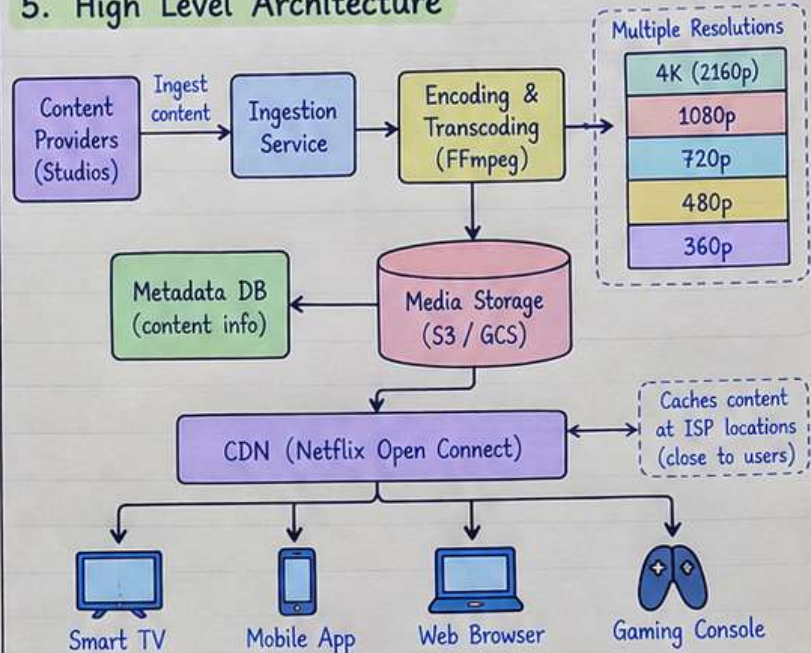
- GET /videos/{id} (get metadata)
- GET /videos/{id}/stream (get stream URL)
- POST /users/{id}/progress (save watch progress)
- GET /users/{id}/history (watch history)
- GET /search?q=... (search content)
- POST /users/{id}/rating (rate a title)

4. Data Model (Simplified)

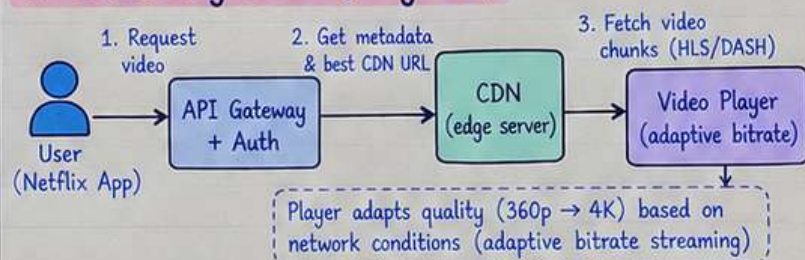
Movies / Shows	User Profile
id (PK)	id (PK)
title	name
description	email
genre	profiles (multiple)
duration	preferences (genre, lang)
thumbnail_url	watch_history
video_urls (by resolution)	created_at
created_at	
Watch Progress	Ratings / Reviews
id (PK)	id (PK)
user_id (FK)	user_id (FK)
video_id (FK)	video_id (FK)
last_position (sec)	rating (1-5)
updated_at	review_text
	created_at

"Data + Personalization = Better Experience" ❤️

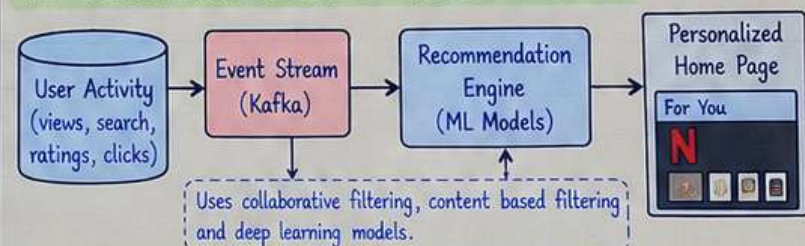
5. High Level Architecture



6. Streaming Flow (Playback)



7. Recommendation & Personalization



8. Key Challenges

- Massive storage (petabytes of video data)
- Efficient transcoding (multiple resolutions)
- Low latency global streaming
- High bandwidth cost
- Caching strategy (CDN / Open Connect)
- Personalization at scale (real-time + batch)
- Handling peak traffic (new releases)
- Content licensing & geo-restrictions
- Preventing piracy (DRM, encryption)
- Maintaining high availability (multi-region)

9. Interview Tips

- Discuss trade-offs: quality vs cost
- Why adaptive bitrate (HLS/DASH)
- How to design CDN (edge caching)
- Talk about recommendation system
- Handling new content ingestion
- Strategies for global scale
- DRM & security considerations
- How to handle live streaming (e.g. Netflix live events)

"It's not just about streaming videos, it's about delivering personalized entertainment at global scale." 😊

6. Uber

(Ride Hailing Platform)

"Move people
Anytime, Anywhere"

1. Requirements

- Rider & driver apps (iOS / Android)
- Real-time ride matching
- Live location tracking
- ETA calculation
- Trip management (start, end, cancel)
- Pricing & payments (surge pricing)
- Ratings & feedback
- High availability & low latency

2. Scale (Rough Estimate)

- Users: 100M+ (global)
- Trips per day: 100M+
- Concurrent active users: 10M+
- Location updates per second: 1M+
- Read : Write ratio = 10 : 1
- Low latency: < 200 ms
- High availability (99.99%)

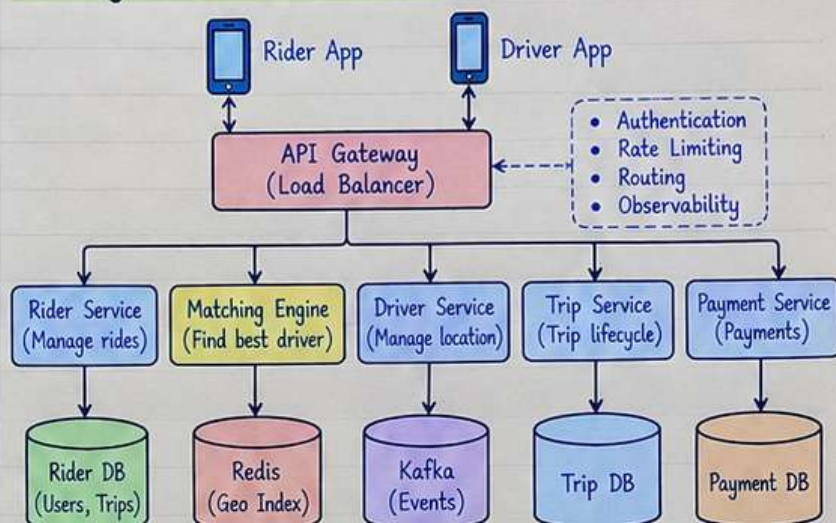
3. API Design (Simplified)

- POST /riders/request (request ride)
- GET /riders/{id}/trips (trip history)
- POST /drivers/location (update location)
- GET /drivers/nearby (find nearby riders)
- POST /trips/{id}/start (start trip)
- POST /trips/{id}/end (end trip)
- GET /trips/{id}/status (track trip)

4. Data Model (Simplified)

Users	Driver Location
id (PK)	id (PK)
name	driver_id (FK)
email	latitude
phone_number	longitude
type (rider/driver)	timestamp
rating	status (active/idle/busy)
created_at	
Trips	Payments
id (PK)	id (PK)
rider_id (FK)	trip_id (FK)
driver_id (FK)	amount
pickup_location	payment_method
drop_location	status (success/failed)
status (requested/ongoing/completed)	transaction_id
fare	created_at
created_at	

5. High Level Architecture

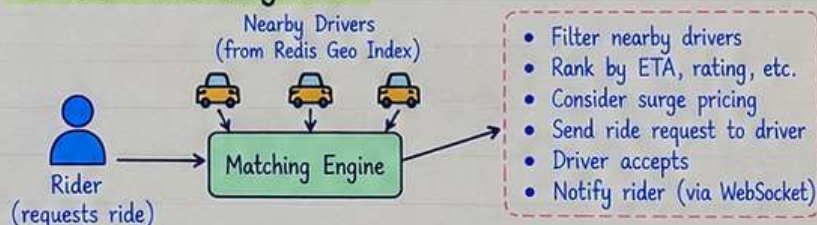


6. Real-time Location Updates Flow

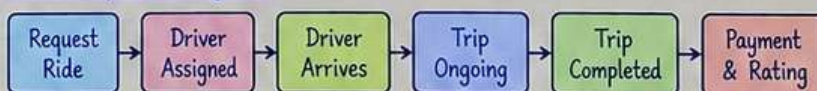


- Drivers send location updates every few seconds (e.g. 5s)
- Store in Redis using GeoHash / Geospatial Index (e.g. Redis GEO)
- Used by Matching Engine to find nearby drivers quickly
- Older data can be moved to cold storage (S3 / Data Lake)

7. Ride Matching Flow



8. Trip Lifecycle



9. Key Challenges

- Real-time matching at massive scale
- Handling millions of location updates / sec
- Low latency (sub-second)
- Accurate ETA calculation (maps, traffic)
- Dynamic pricing (surge)
- Consistency (avoid double booking)
- Fraud detection (fake trips, location spoofing)
- High availability (global operations)

10. Interview Tips

- Explain how you'd design the matching algorithm (e.g. GeoHash)
- Discuss WebSocket vs polling
- Talk about trade-offs (latency vs accuracy)
- How to handle spike in demand (autoscaling, queues)
- Mention real-world systems (Google Maps, Redis, Kafka)

"Good system design gets people
where they need to be." 😊

"It's not just a ride,
it's a real-time system." 😊

7. Twitter / X

(Microblogging Social Platform)



"What's happening in the world, right now"

1. Requirements

- Post short messages (tweets)
- Follow / unfollow users
- View home timeline (tweets from followed users)
- Like, retweet, reply
- Notifications
- Search tweets, hashtags, users
- Handle trending topics
- High availability & low latency
- Support media (images, videos, GIFs)

2. Scale (Rough Estimate)

- Users : 500M+ (global)
- Tweets per day : 500M+
- Read : Write ratio = 100 : 1
- Avg tweet size : ~ 1 KB (text)
- Media tweets : ~ 1 - 10 MB
- Concurrent users : 10M+
- High availability (99.99%)
- Low latency : < 200 ms

3. API Design (Simplified)

- POST /tweets (create tweet)
- GET /tweets/{id} (get tweet)
- GET /users/{id}/timeline (home feed)
- GET /users/{id}/tweets (user tweets)
- POST /tweets/{id}/like
- GET /tweets/{id}/retweet
- POST /users/{id}/follow
- GET /search?q=... (search tweets)

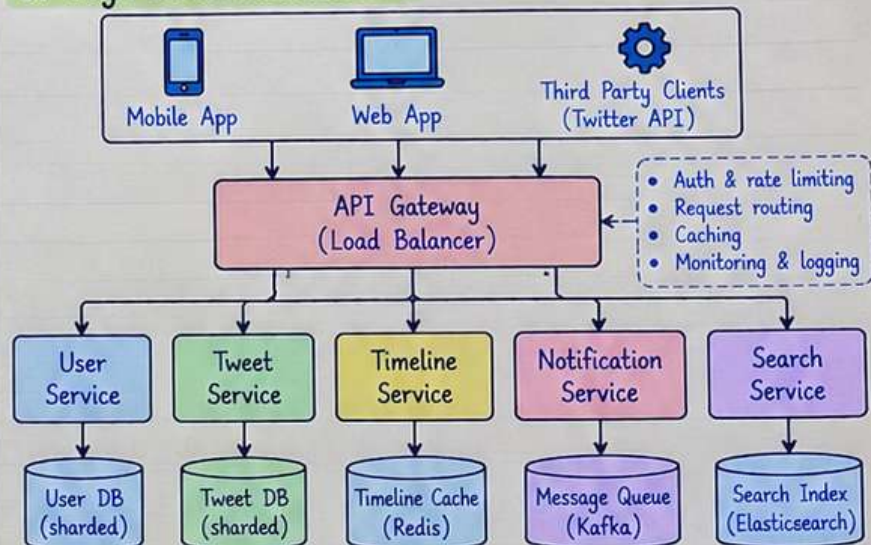
4. Data Model (Simplified)

Users	Tweets
id (PK)	id (PK)
username	user_id (FK)
email	content
name	media_url
profile_pic_url	like_count
created_at	retweet_count
	created_at
Follows	Likes
id (PK)	id (PK)
follower_id (FK)	user_id (FK)
followee_id (FK)	tweet_id (FK)
created_at	created_at

9. Scaling Strategies

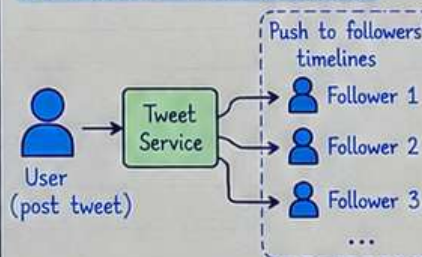
- Use caching (Redis, CDN)
- Sharding (tweets, users, timelines)
- Kafka for asynchronous processing
- Separate read & write databases
- Use fan-out hybrid model
- Partition data by user_id
- Auto-scaling stateless services
- Multi-region deployment

5. High Level Architecture



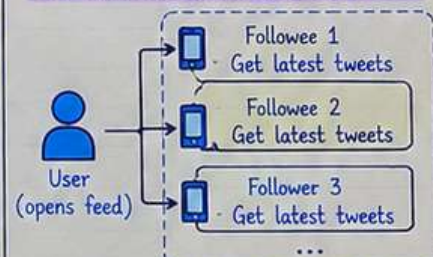
6. Feed Generation Strategies

1. Fan-out on Write (Push)



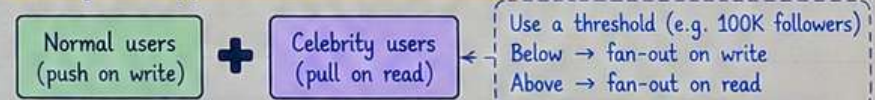
- When a user posts, push to all followers' timelines (pre-compute)
- Fast read, heavy write
- Good for normal users (< 100K followers)

2. Fan-out on Read (Pull)



- When a user opens feed, fetch latest tweets from followees
- Slower read, less write
- Good for celebrity accounts (millions of followers)

3. Hybrid Approach (Used in Twitter)



7. Other Components

- Timeline caching (Redis)
- Media storage (S3 + CDN)
- Search (Elasticsearch)
- Trending (real-time + batch)
- Notifications (Kafka + consumers)
- Spam & abuse detection
- Analytics (Hadoop/Spark)
- Content moderation
- Global distribution (multi-region)

8. Key Challenges

- Handling massive read traffic
- Celebrity problem (hot users)
- Low latency feed generation
- Real-time trending topics
- Spam, abuse & fake accounts
- Tweet ranking & relevance
- Data consistency
- Media storage & bandwidth
- High availability (global scale)

10. Interview Tips

- Discuss trade-offs (push vs pull)
- How to handle celebrity accounts?
- Talk about timeline ranking (ML)
- Discuss caching strategy
- Mention real-world optimizations (sharding, queues, CDN)

"A simple tweet can reach millions – that's the power of a well-designed system."



8. Google Drive

(Distributed File Storage & Sync)



"Store. Sync.
Share. Access Anywhere."

1. Requirements

- Upload, download and store files
- Organize files in folders (hierarchy)
- Sync across multiple devices
- File sharing with permissions
- Handle large files (GBs)
- Versioning & restore previous versions
- Real-time or near real-time sync
- High availability & durability
- Support different file types
- Deduplication (avoid storing same file)

2. Scale (Rough Estimate)

- Users: 2B+ (global)
- Files stored: 100B+
- Daily uploads: 1B+
- Avg file size: 10 MB – 1 GB
- Total storage: Exabytes (EB)
- Concurrent users: 10M+
- High availability: 99.99%
- Low latency (metadata): < 100 ms

3. API Design (Simplified)

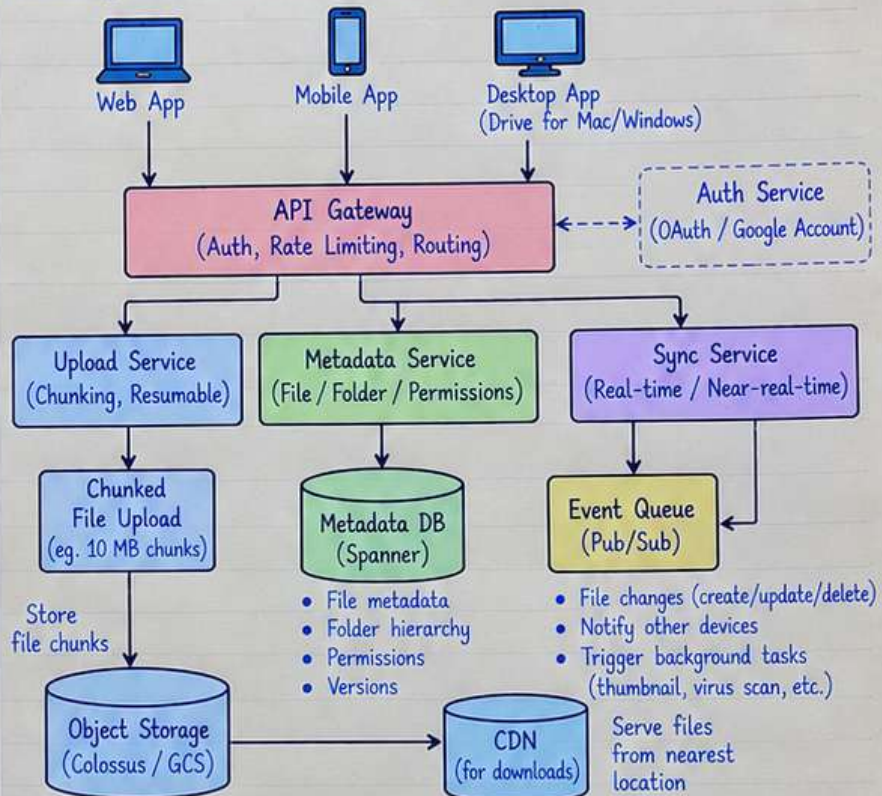
- POST /files (upload file - multipart)
- GET /files/{id} (get metadata)
- GET /files/{id}/download (download)
- PUT /files/{id} (update metadata)
- GET /files/{id}/versions (list versions)
- POST /files/{id}/share (share file)
- GET /files (list files with filters)
- POST /folders (create folder)
- GET /search?q=... (search files)

4. Data Model (Simplified)

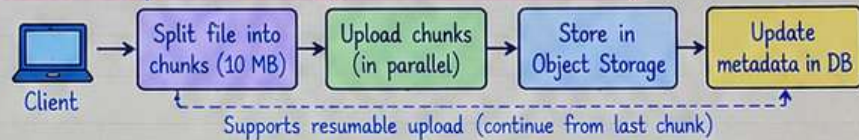
Files	File Versions
id (PK) name mime_type size owner_id (FK) parent_id (FK) current_version_id created_at updated_at is_deleted	id (PK) file_id (FK) version_number storage_path size checksum (hash) created_at created_by
Users	Shares
id (PK) name email storage_quota created_at	id (PK) file_id (FK) shared_with (user_id/email) permission (view/edit) created_at

Metadata tells us about the file.
File data is stored separately in distributed storage.

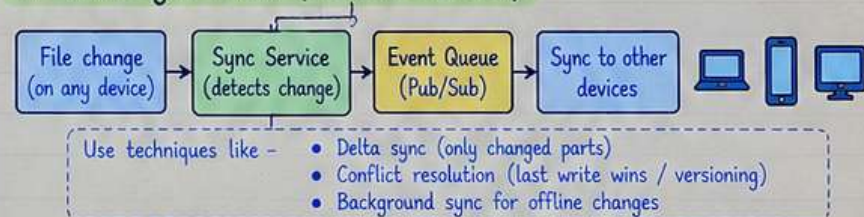
5. High Level Architecture



6. File Upload Flow (Chunked & Resumable)



7. File Sync Flow (Across Devices)



8. Key Challenges

- Storing massive amounts of data (EBs)
- Handling large file uploads (resumable)
- Data consistency across devices
- File versioning & conflict resolution
- Low latency metadata operations
- Reliable and durable storage
- Deduplication (same file uploaded multiple times)
- Security & access control (sharing permissions)
- Virus/malware scanning
- High availability (global scale)
- Efficient sync (battery & bandwidth)

9. Scaling Strategies

- Distributed storage (GCS / Colossus)
- Globally distributed metadata (Spanner)
- CDN for fast downloads
- Chunked and parallel uploads
- Caching metadata (e.g. in Redis)
- Event-driven architecture (Pub/Sub)
- Sharding by user_id / file_id
- Deduplication using file hash (MD5/SHA)
- Multi-region replication
- Auto-scaling stateless services

10. Interview Tips

- Explain why metadata and file data are stored separately
- Discuss trade-offs in consistency vs availability
- How to handle very large files and resumable uploads
- Talk about sync conflict resolution
- Mention security (encryption, access control, virus scan)

"Your files, everywhere. That's the power of a well-designed system."



9. E-commerce / Amazon

(Online Shopping Platform)

"From search to doorstep" 🛒

Billions of products
Millions of happy customers

1. Requirements

- Browse & search products
- View product details (images, reviews)
- Add to cart and checkout
- Manage inventory
- Process payments (multiple methods)
- Track orders (real-time)
- Handle returns & refunds
- Send notifications (email, SMS, push)
- High availability, low latency
- Support massive traffic (sales events)

2. Scale (Rough Estimate)

- Users : 500M+ (global)
- Products : 100M+
- Orders per day : 10M+
- Peak QPS : 100K+ (during sales)
- Avg product page size : ≈ 100 KB
- Read : Write ratio : 10 : 1
- High availability (99.99%)
- Global scale (multi-region)

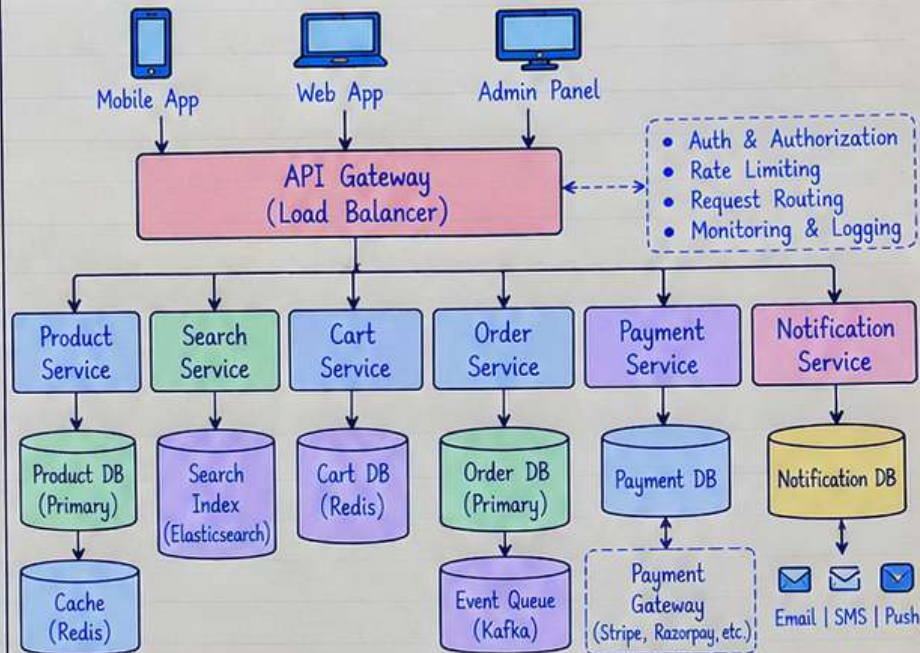
3. API Design (Simplified)

- GET /products?q=... (search)
- GET /products/{id} (product details)
- POST /cart/items (add to cart)
- GET /cart (get user cart)
- POST /orders (place order)
- GET /orders/{id} (order details)
- POST /payments (process payment)
- POST /returns (initiate return)

4. Data Model (Simplified)

Products	Orders
id (PK) name description price category_id brand inventory_count ratings created_at	id (PK) user_id (FK) total_amount status (CREATED, PAID, SHIPPED, DELIVERED) payment_id (FK) created_at updated_at
Users	Order Items
id (PK) name email password_hash address created_at	id (PK) order_id (FK) product_id (FK) quantity price created_at

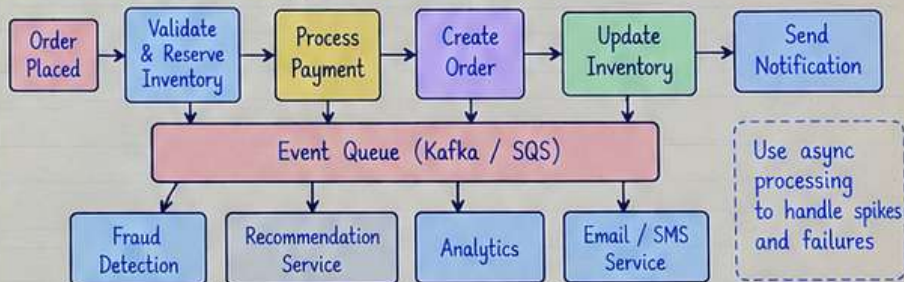
5. High Level Architecture



6. Checkout Flow



7. Order Processing Flow (Async)



8. Key Challenges

- Inventory consistency (avoid overselling)
- Handling high traffic (e.g. sales, Black Friday)
- Reliable payment processing
- Distributed transactions (order + payment)
- Low latency search at scale
- Caching strategy (products, user sessions, cart)
- Fraud detection (fake orders, bots)
- Handling returns & refunds
- Data consistency across services
- Global scale & multi-region deployment
- High availability (99.99%)

9. Scaling Strategies

- Use CDN for static assets (images, CSS, JS)
- Cache product data (Redis)
- Search with Elasticsearch / OpenSearch
- Partition databases (by product_id, user_id)
- Use message queues (Kafka) for async tasks
- Auto-scaling during traffic spikes
- Use microservices (independent scaling)
- Read replicas for databases
- Use NoSQL for high write workloads (cart)
- Multi-region deployment
- Implement circuit breakers & retries

10. Interview Tips

- Discuss trade-offs: strong vs eventual consistency
- How would you handle inventory reservation?
- How to design search at scale?
- How to handle failed payments?
- Talk about real-world optimizations (CDN, caching, queues)



"Good data model = Reliable orders"

"Never sell what you don't have."

"Inventory consistency is critical!"



10. Google Maps

(Maps, Search & Navigation Platform)



"Helping billions find their way in the real world"

1. Requirements

- Search for places (restaurants, ATMs, etc.)
- View maps (road, satellite, terrain)
- Get directions (car, bike, walk, transit)
- Real-time traffic updates
- Estimate ETA (accurate & dynamic)
- Support live location sharing
- Find nearby places (e.g. coffee near me)
- Handle billions of queries
- High availability & low latency
- Support multiple platforms (iOS, Android, Web)

2. Scale (Rough Estimate)

- Users : 1B+ (global)
- Queries per day : 100B+
- Map tiles served per day : 10B+
- Location updates per second : 1M+
- Places in database : 200M+
- Avg response time : < 200 ms
- High availability (99.99%)
- Global scale (multi-region)

3. API Design (Simplified)

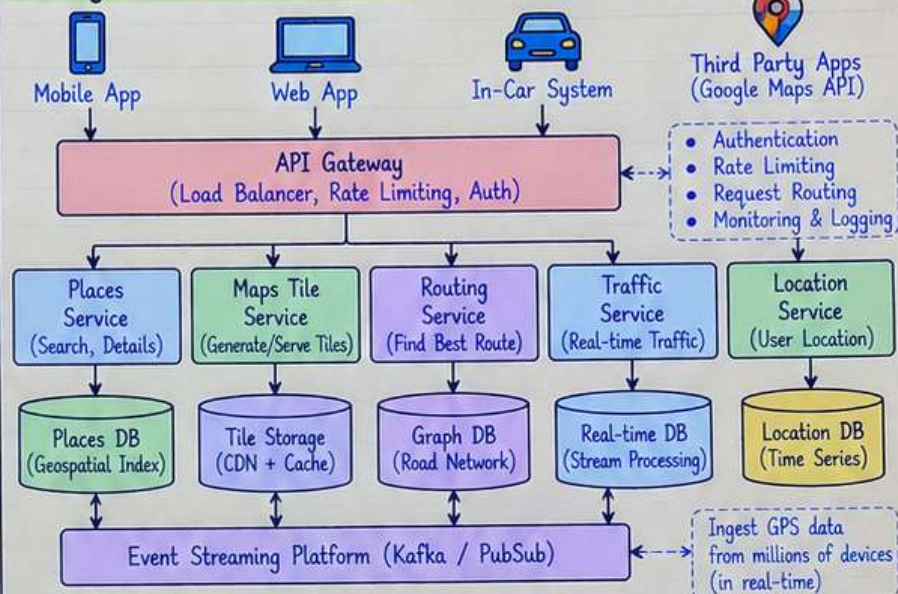
- GET /maps/tiles/{z}/{x}/{y} (get map tile)
- GET /places/search?query=... (search places)
- GET /places/nearby?lat=...&lng=... (nearby)
- GET /directions?source=...&dest=... (get route)
- GET /directions/eta (get ETA)
- POST /location/update (update user location)
- GET /place/{id} (place details)

4. Data Model (Simplified)

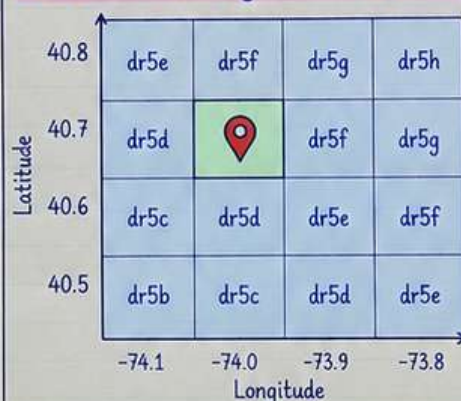
Places	Roads / Graph
id (PK)	id (PK)
name	source_node_id
category	target_node_id
location (lat, lng)	distance
address	speed_limit
rating	road_type
opening_hours	geometry (polyline)
created_at	created_at
updated_at	updated_at

Users	Location Logs
id (PK)	id (PK)
name	user_id (FK)
email	lat
current_location	lng
saved_places	timestamp
created_at	device_info

5. High Level Architecture



6. Find Nearby Places (Geospatial Indexing)



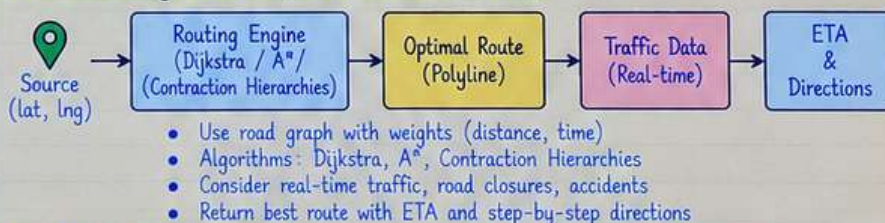
GeoHash / QuadTree

- Divide the world into grids
- Each cell has a unique key (e.g. dr5e)
- Store places in corresponding cells
- For "nearby" search, query current cell + adjacent cells
- Supports fast geospatial search at scale

Example:

- Find restaurants near (40.7, -73.9)
- Get geohash for current location
 - Query current cell + nearby cells
 - Rank results by distance, rating, etc.

7. Routing Flow (Directions & ETA)



8. Key Challenges

- Storing and querying massive geospatial data
- Real-time traffic processing (millions of updates/sec)
- Low latency for map tiles and search
- Accurate and dynamic ETA calculation
- Handling global scale and high availability
- Frequent map updates (new roads, businesses)
- Dealing with noisy GPS data (accuracy issues)
- Optimizing storage and caching for map tiles
- Supporting offline maps
- Privacy and data security (location data)

9. Scaling Strategies

- Use GeoHash / QuadTree for geospatial index
- CDN for map tiles (static content)
- Distributed graph database for road network
- Stream processing (Kafka) for real-time traffic
- Caching (Redis) for popular places & tiles
- Partition data by region (country/city)
- Use hierarchical tiles (different zoom levels)
- Multi-region deployment
- Batch + real-time data pipelines
- Auto-scaling based on query load

10. Interview Tips

- How would you design a geospatial index from scratch?
- Which routing algorithm would you use and why?
- How to handle real-time traffic updates?
- Discuss trade-offs: accuracy vs latency
- How to store and serve map tiles efficiently?
- How to handle billions of location updates?

"Location data + real-time updates enable a smarter, more connected world."

"Maps are not just about places, but about possibilities."

11. Food Delivery

(On-Demand Food Delivery Platform)

“Good food at your doorstep” 😊

1. Requirements

- Customer app (search, order, track)
- Restaurant app (manage menu, orders)
- Delivery partner app (accept, navigate)
- Real-time order tracking (live location)
- Multiple payment methods (UPI, Card, COD)
- Search nearby restaurants
- Handle peak loads (lunch/dinner time)
- Ratings & reviews
- Notifications (order, ETA, updates)
- High availability & low latency
- Support cancellations, refunds

2. Scale (Rough Estimate)

- Users : 100M+ (global)
- Restaurants : 5M+
- Orders per day : 10M+
- Peak orders per second : 5K - 10K
- Avg order value : \$5 - \$20
- Delivery partners : 1M+
- Read : Write ratio = 10 : 1
- High availability (99.99%)
- Low latency : < 200 ms

3. API Design (Simplified)

- GET /restaurants/nearby?lat=...&lng=...
- GET /restaurants/{id} (menu, details)
- GET /search?q=... (search restaurants)
- POST /cart/items (add to cart)
- POST /orders (place order)
- GET /orders/{id} (order details)
- POST /orders/{id}/cancel (cancel order)
- GET /delivery/track/{id} (live location)
- POST /restaurants/{id}/menu (update menu)

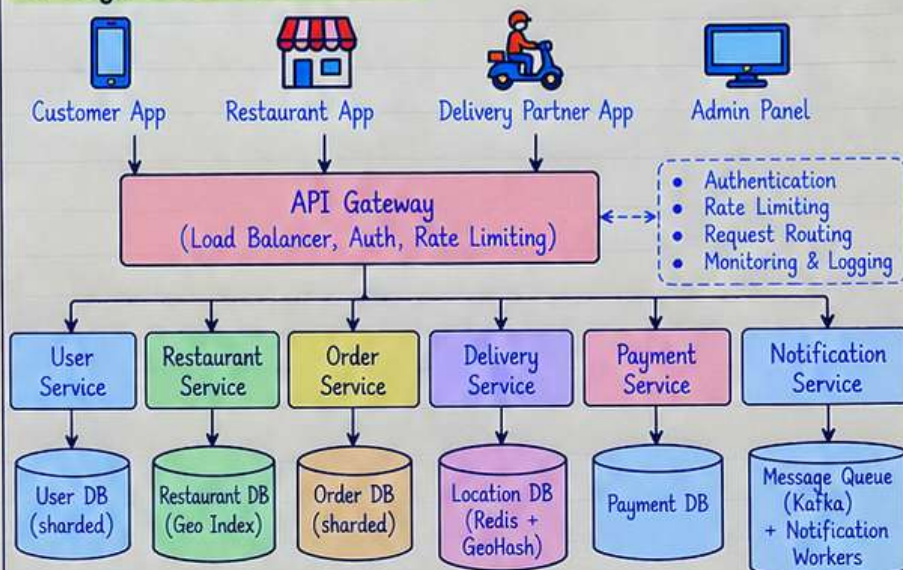
4. Data Model (Simplified)

Users	Restaurants
id (PK)	id (PK)
name	name
email	cuisine_type
phone	location (lat, lng)
address	address
type (customer/partner)	rating
rating	is_active
created_at	created_at

Orders	Menu Items
id (PK)	id (PK)
user_id (FK)	restaurant_id (FK)
restaurant_id (FK)	name
partner_id (FK)	description
status	price
total_amount	is_available
created_at	category
updated_at	created_at

“Great system design delivers food and happiness.” 😊

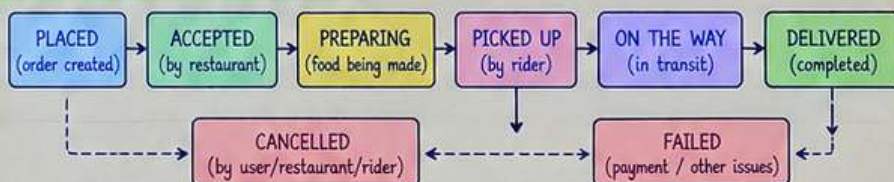
5. High Level Architecture



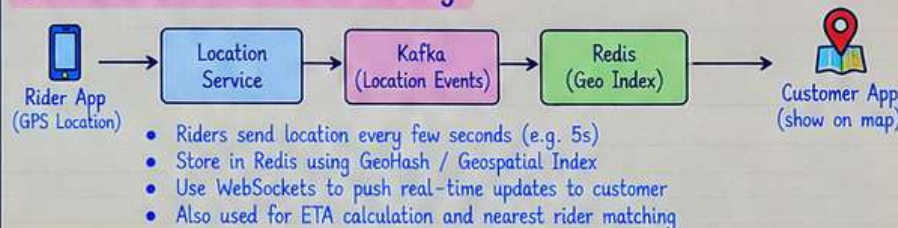
6. Order Flow



7. Order State Machine



8. Real-time Location Tracking



9. Key Challenges

- Real-time matching of orders and riders
- Accurate ETA calculation (traffic, distance, etc.)
- Handling high traffic during peak hours
- Inventory/menu updates and availability
- Cancellations, refunds and fraud prevention
- Consistent order state across services
- Low latency for search and nearby restaurants
- Scalability (global operations)
- Reliable real-time tracking
- High availability (99.99%)

10. Scaling Strategies

- Use Redis for caching (restaurants, menu, nearby)
- Geospatial indexing (GeoHash) for location queries
- Kafka for event-driven processing (orders, tracking)
- Auto-scaling microservices (Kubernetes)
- CDN for static assets (images, app content)
- Read replicas for read-heavy data (search, menus)
- Partition data by region/city
- Use WebSockets for real-time updates
- Implement circuit breakers and retries

11. Interview Tips

- How would you design the rider matching algorithm?
- How to handle peak traffic (e.g. during dinner time)?
- How to calculate ETA accurately?
- How to handle cancellations and refunds?
- Discuss trade-offs: consistency vs availability.

“From restaurants to your door – powered by great engineering.” 😊

12. Slack / Real-Time Chat

(Real-Time Messaging Platform)



"Better conversations build better teams."

1. Requirements

- Real-time messaging (1:1 and channels)
- Create / join / leave channels
- Send text, files, emojis, reactions
- Show online presence (active, idle, offline)
- Message history & search
- Push notifications (offline users)
- Edit / delete messages
- Threaded conversations
- High availability & low latency
- Support large teams (10K+ members)

2. Scale (Rough Estimate)

- Users : 100M+ (global)
- Daily active users : 10M+
- Messages per day : 10B+
- Concurrent connections : 1M+
- Avg message size : ≈ 1 KB
- File uploads : 100M+ per day
- Read : Write ratio : 10 : 1
- High availability (99.99%)
- Low latency : < 100 ms

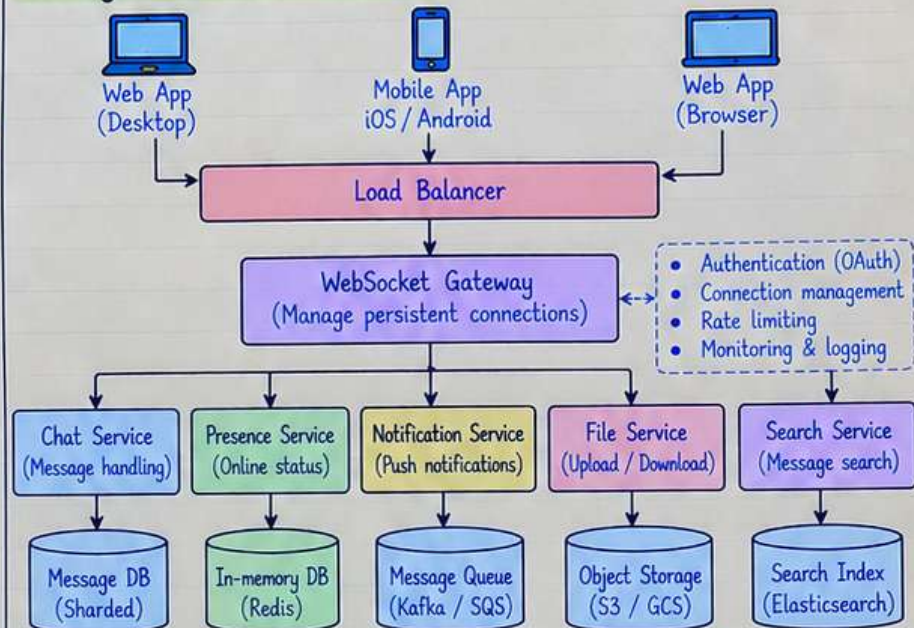
3. API Design (Simplified)

- POST /messages (send message)
- GET /channels (list channels)
- GET /channels/{id}/messages (get history)
- POST /channels/{id}/join (join channel)
- POST /channels/{id}/leave (leave channel)
- GET /users/{id}/presence (get status)
- POST /files (upload file)
- GET /search?q=... (search messages)

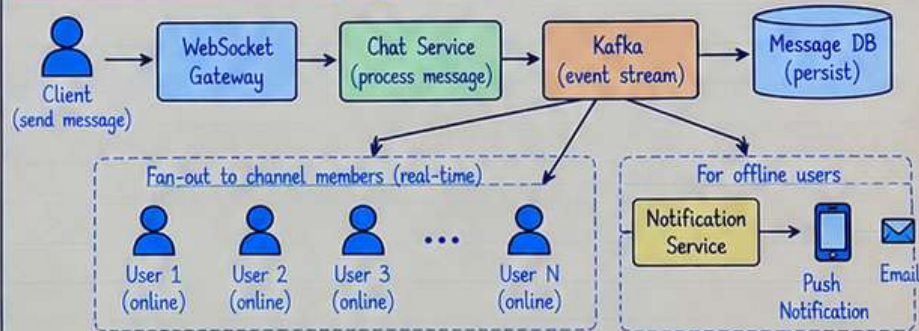
4. Data Model (Simplified)

Users	Channels
id (PK) name email password_hash avatar_url status (online/idle/offline) created_at	id (PK) name type (public/private) created_by (FK) created_at
Messages	Files
id (PK) channel_id (FK) user_id (FK) content message_type (text/file) timestamp edited_at deleted_at	id (PK) user_id (FK) channel_id (FK) file_url file_type file_size created_at

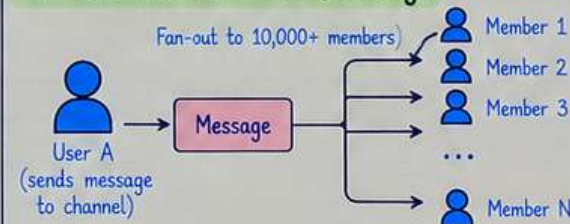
5. High Level Architecture



6. Real-Time Messaging Flow



7. Channel Fan-out (Scaling)



Strategies:

- Use Kafka for fan-out
- Store messages in DB
- Push to online users via WebSockets
- Use push notifications for offline users
- Partition by channel_id

8. Key Challenges

- Handling millions of concurrent connections
- Real-time message delivery (low latency)
- Message ordering & deduplication
- Scaling channel fan-out (large channels)
- Presence status at scale
- Fault tolerance & reconnect handling
- Message search (full text)
- File storage & virus scanning
- Data consistency across regions
- High availability (99.99%)

9. Scaling Strategies

- Use WebSockets for real-time communication
- Kafka for asynchronous processing
- Redis for presence and caching
- Partition data by channel_id / user_id
- Use CDN for file downloads
- Elasticsearch for fast search
- Auto-scaling stateless services
- Multi-region deployment
- Implement backpressure and rate limiting

10. Interview Tips

- Discuss trade-offs: WebSockets vs long polling
- How to handle message ordering?
- How to scale large channels (10K+ members)?
- How to handle offline users?
- Talk about presence system design
- Discuss failure scenarios and recovery
- Mention optimizations (compression, batching, etc.)

"Real-time communication isn't just about technology, it's about bringing people closer."



13. Real-Time Location Tracking

(Track Millions of Devices in Real-Time)

"Location connects the real world."

1. Requirements

- Track real-time location of millions of devices
- Ingest GPS updates (lat, long, timestamp)
- Show live location on map (near real-time)
- Find nearby devices (e.g. nearby drivers)
- Low latency (sub-second)
- Handle high write throughput
- Store historical location data (analytics)
- Location freshness (avoid stale data)
- Support different device types (mobile, IoT, vehicles)
- High availability & durability
- Scalable to global regions

2. Scale (Rough Estimate)

- Devices : 10M+ (global)
- Location updates per device : 1 / 5 sec
- Total updates per second : ~ 2M+
- Daily location records : 100B+
- Data size per record : ~ 50 bytes
- Daily data ingestion : ~ 5 TB+
- Read : Write ratio : 1: 10 (mostly writes)
- Query for nearby devices : 100M+ per day
- High availability (99.99%)
- Low latency : < 200 ms

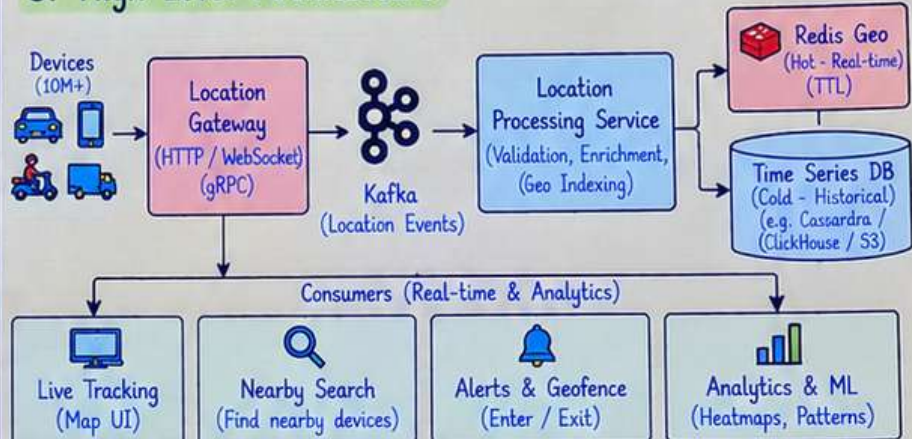
3. API Design (Simplified)

- POST /location/update (send location)
- GET /location/{deviceId} (get latest)
- GET /nearby?lat=...&lng=...&radius=... (find nearby devices)
- GET /history/{deviceId}?from=...&to=... (get history)
- GET /geofence/{id} (get geofence details)
- POST /geofence (create geofence)

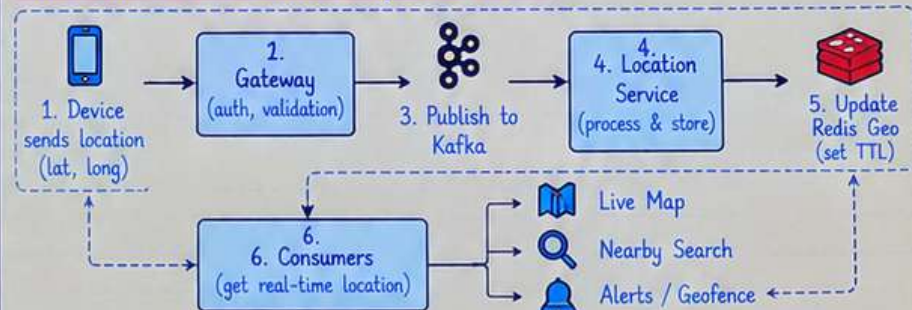
4. Data Model (Simplified)

Location Records (Hot Data - in Redis)	Historical Records (Cold Data)
device_id (PK) latitude longitude timestamp speed heading metadata (JSON) ttl (expire old data)	id (PK) device_id latitude timestamp speed event_type region created_at
Geofence	Users / Devices
id (PK) name polygon (list of coordinates) type (circle / polygon) created_at	id (PK) device_id (unique) type (driver / user / IoT) owner_id status (active / inactive) created_at

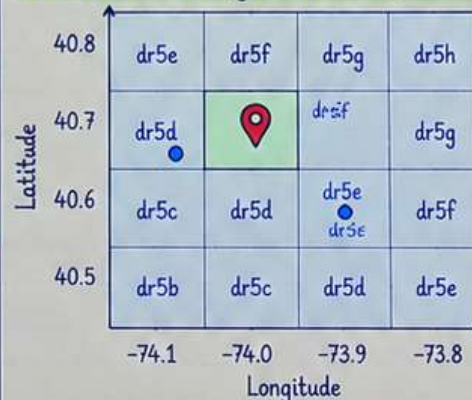
5. High Level Architecture



6. Real-Time Location Update Flow



7. Find Nearby Devices (GeoHash)



How it works?

- Divide world into grids using GeoHash
- Store device location in corresponding cell
- To find nearby devices, search current cell + adjacent cells
- Filter by distance (Haversine formula)
- Supports fast and scalable geospatial queries

Example :

- Find nearby drivers within 2 km
- Get geohash for user's location
 - Query current cell + adjacent cells
 - Filter by actual distance
 - Return top N nearest drivers

8. Key Challenges

- Handling millions of location updates per second
- Low latency for real-time queries
- Stale location data (handle with TTL)
- Ordering of location events
- Accurate geospatial queries at scale
- Data storage & retention (hot vs cold data)
- High availability and fault tolerance
- Dealing with network issues / missing updates
- Privacy and data security (location is sensitive)
- Cost optimization for large data volume

9. Scaling Strategies

- Use Kafka for high-throughput ingestion
- Store latest location in Redis (Geo + TTL)
- Use GeoHash / S2 for geospatial indexing
- Partition data by geohash or region
- Use time-series DB for historical data
- Asynchronous processing for analytics
- Use WebSockets for live updates
- Multi-region deployment (global scale)
- Implement backpressure and rate limiting
- Use CDN for map tiles (if building your own map)

10. Interview Tips

- How to handle out-of-order location updates?
- What if a device stops sending updates?
- How to design for global scale?
- How to efficiently query nearby devices?
- How to store location history for analytics?
- Discuss trade-offs: accuracy vs latency vs cost.
- How to handle a sudden spike in location updates?



"Location data is a stream, not just a database record."

"Real-time location turns data into meaningful, real-world experiences."



14. Recommendation System

(Personalized Recommendations at Scale)

"Right content to the right user at the right time."

(Netflix, YouTube, Instagram, Amazon, Spotify, etc.)

1. Requirements

- Show personalized recommendations (e.g. videos, products, posts, songs)
- Handle millions of users and items
- Real-time and batch recommendations
- Support different use cases (home feed, search, related items)
- Low latency (< 200 ms)
- High availability and scalability
- Continuous learning from user feedback
- Handle cold start (new users / new items)
- Diverse and relevant results
- A/B testing and experiment support

2. Scale (Rough Estimate)

- Users : 100M+ (global)
- Items (videos/products) : 100M+
- Events per day (clicks, views, likes) : 10B+
- Read QPS : 1M+ (recommendation requests)
- Write QPS : 100K+ (user events)
- Avg response time : < 200 ms
- Data per event : ~ 1 KB
- Data per day : 10+ TB
- High availability (99.99%)
- Multi-region deployment

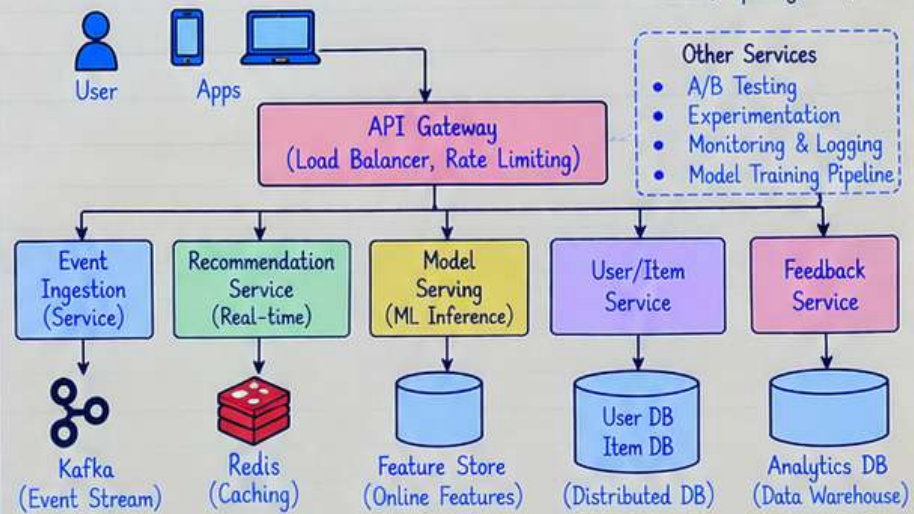
3. API Design (Simplified)

- POST /events (log user event)
- GET /recommendations?user_id=...
- GET /items/{id}/similar (similar items)
- GET /users/{id}/history (user history)
- POST /feedback (explicit feedback)
- GET /trending (trending items)

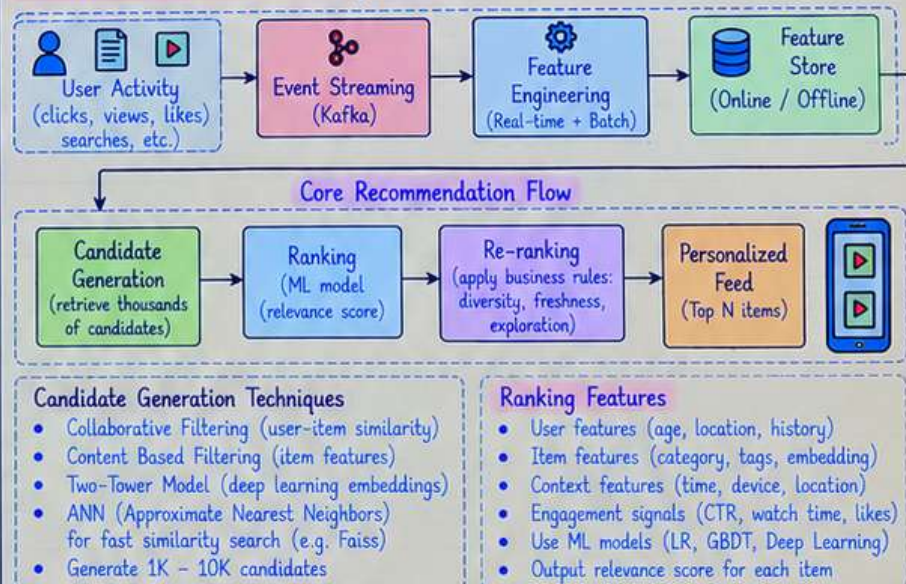
4. Data Model (Simplified)

User Events	Items
id (PK) user_id (FK) item_id (FK) event_type (view, click, like) timestamp metadata (JSON) created_at	id (PK) title description category_id tags content_type metadata (JSON) created_at
Users	Item Features
id (PK) name email country device_type signup_date created_at	item_id (PK) category tags embedding_vector popularity_score created_at updated_at

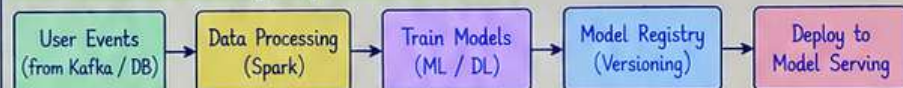
5. High Level Architecture



6. Recommendation Pipeline



7. Model Training Pipeline (Offline)



8. Key Challenges

- Handling massive scale (users, items, events)
- Low latency for real-time recommendations
- Cold start problem (new users / new items)
- Keeping recommendations fresh
- Balancing relevance, diversity and novelty
- Handling changing user interests
- Feature engineering at scale
- A/B testing and online experiments
- Preventing bias and ensuring fairness
- High availability (99.99%) and multi-region

9. Scaling Strategies

- Use Kafka for event ingestion (high throughput)
- Use Redis / CDN for caching recommendations
- Use ANN (Faiss) for fast similarity search
- Partition data by user_id / item_id
- Use distributed feature store (e.g. Feast)
- Model serving with autoscaling (Kubernetes)
- Multi-region deployment
- Use batch + real-time hybrid pipeline
- Optimize models (quantization, batching)
- Monitor model performance and drift

10. Interview Tips

- How would you solve cold start?
- What features would you use for ranking?
- How to ensure diversity in recommendations?
- How to handle real-time vs batch processing?
- How would you measure recommendation quality?
- Discuss trade-offs: accuracy vs latency vs scalability.



"Data + ML + Scale = Better Experiences for Billions of Users."



"Personalization is not a feature, it's a product advantage."



15. Google Search

(Crawl, Index and Search the Web at Scale)

"Organizing the world's information and making it universally accessible."

1. Requirements

- Crawl and index billions of web pages
- Return relevant results for any query
- Support text, images, videos, news, etc.
- Low latency (< 200 ms)
- Handle real-time content and freshness
- Autocomplete and search suggestions
- Handle high QPS and global traffic
- Deduplicate and avoid spam content
- Support multiple languages and regions
- High availability and fault tolerance

2. Scale (Rough Estimate)

- Web pages : 100B+ (indexed)
- New/updated pages per day : 1B+
- Queries per day : 10B+
- Peak QPS : 1M+
- Avg query size : < 100 bytes
- Avg search results : top 10 (of millions)
- Index size : 100s of TBs → PBs
- Crawl bandwidth : multi-Tbps
- High availability (99.99%)
- Global scale (multi-region)

3. API Design (Simplified)

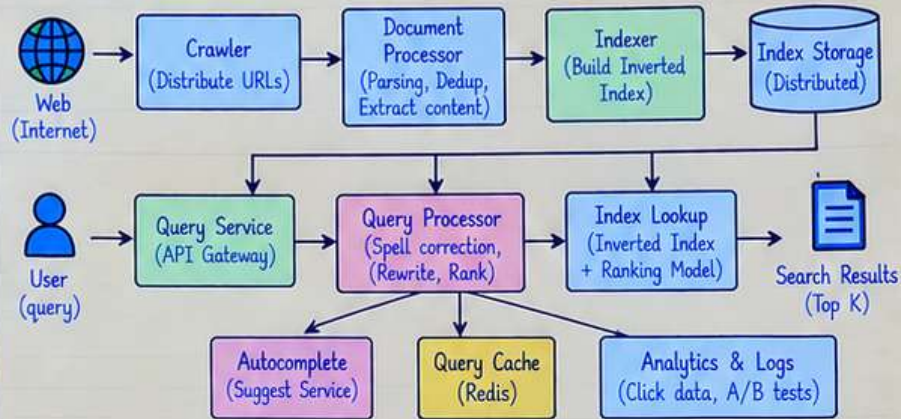
- GET /search?q=... (search results)
- GET /suggest?q=... (autocomplete)
- GET /page?id=... (cached page)
- GET /image-search?q=...
- GET /news?q=...
- GET /video-search?q=...

4. Data Model (Simplified)

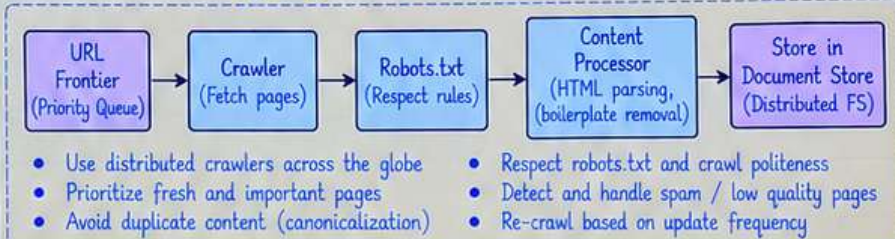
Document	Inverted Index
id (PK)	term (PK)
url	doc_id (PK)
content	term_frequency
content_type (html, pdf, ...)	positions
language	metadata
last_crawled_at	
metadata (JSON)	

Link Graph	Page Metadata
source_url (PK)	id (PK)
target_url (PK)	title
anchor_text	description
link_type	keywords
discovered_at	last_updated_at
	page_rank
	is_duplicate

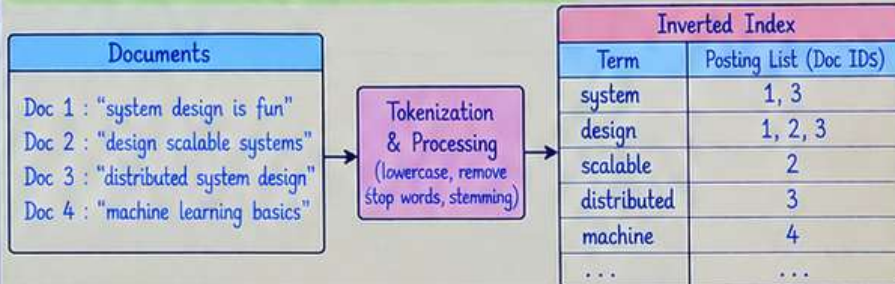
5. High Level Architecture



6. Web Crawling Pipeline



7. Inverted Index (How Search Works)



8. Query Flow (End to End)



8. Key Challenges

- Crawling and indexing billions of pages
- Handling real-time updates and freshness
- Ranking high quality and relevant results
- Dealing with spam, duplicate and low quality content
- Low latency at massive scale
- Storing and querying huge inverted index
- Supporting multiple languages and content types
- Global distribution and fault tolerance

10. Scaling Strategies

- Distributed crawlers and indexers (parallelism)
- Partition/shard index by term (range/hash)
- Use CDN and edge servers for low latency
- Query caching (Redis / in-memory)
- Data compression for index storage
- Replica and multi-region deployment
- Use ML models for better ranking
- Incremental updates and near real-time indexing
- Monitor, A/B test and continuously improve

11. Interview Tips

- How would you handle fresh content and re-crawling?
- How to design an inverted index at massive scale?
- Discuss ranking factors (PageRank, relevance, ML).
- How to handle duplicate and low quality content?
- What optimizations ensure low latency?
- How would you scale for global traffic?



"Crawl the web, index the knowledge, serve the world."



"Search is not just about finding pages, but about finding the right answers."

